

7.1 模型 vs Agent的结构化输出对比

第06章已经介绍过结构化输出，当时的重点是与模型对象的绑定，这里与Agent的结构化输出做对比：

维度	模型的结构化输出	Agent 结构化输出
操作对象	作用于 大模型对象	作用于 Agent
解析时机	每次模型调用生成 <code>AIMessage</code> 时，进行解析	仅在 Agent 决定「任务结束」并输出最终答案时解析
数据流转	模型 → 结构化对象	模型 → 工具 → 反思 → … → 结构化对象
绑定方式	使用 <code>with_structured_output</code>	使用 <code>response_format</code> 参数
适用场景	单次、确定性 的任务（如提取字段、翻译、分类）	多步、复杂推理 的任务（如查文档后汇总报表）

7.2 结构化输出的4种策略

LangChain的`create_agent()`函数自动处理结构化输出的全过程。用户只需通过“`response_format`”参数设置期望的输出模式（Schema）。当模型生成结构化数据时，系统会自动捕获、验证并将结果存储在Agent状态的`structured_response`键中。

```
In [ ]: def create_agent(  
    ...  
    response_format: Union[  
        ToolStrategy[StructuredResponseT],  
        ProviderStrategy[StructuredResponseT],  
        type[StructuredResponseT],  
        None,  
    ]
```

`create_agent`函数中的`response_format`参数支持四种不同的策略设置方式：

① ProviderStrategy

使用模型提供商的原生结构化输出功能实现结构化输出。

- 这里所说的“原生结构化输出”指的是大语言模型（LLM）提供商通过其API直接提供的、在模型响应阶段就强制保证输出格式符合预定规范的能力，这种能力能够在模型生成内容的源头确保结构化准确性。
- 适用于支持原生结构化输出的模型，比如**OpenAI、Anthropic Claude或xAI Grok**等。

注意“

1. **deepseek-v4-flash** 不支持 OpenAI 新标准 **json_schema** 模式
init_chat_model 默认 with_structured_output(method="json_schema")，会传递 response_format: {"type": "json_schema", ...}，DeepSeek API 直接拒绝。
2. 同时 deepseek-v4-flash 开启思考模式（reasoner）时，**function_calling** + **tool_choice="required"** 也会 400 报错。
3. DeepSeek 仅支持基础模式：response_format={"type": "json_object"}（只保证合法 JSON，不强制校验 Schema）。

举例：

```
In [ ]: from pydantic import BaseModel, Field
        from langchain.agents import create_agent
        from langchain.agents.structured_output import ProviderStrategy
        from langchain.messages import HumanMessage
        from rich import print as rprint
        from langchain.chat_models import init_chat_model
        from dotenv import load_dotenv
        import os

        load_dotenv(override=True)

        # 1. 模型初始化
        model = init_chat_model(
            model="deepseek-v4-flash",
            model_provider="deepseek",
            api_key=os.getenv("DEEPSEEK_API_KEY"),
            base_url=os.getenv("DEEPSEEK_BASE_URL")
        )

        # 2. Pydantic 结构化方式定义
        class ContactInfo(BaseModel):
            """用户的联系方式"""
            name: str = Field(description="用户姓名")
            email: str = Field(description="用户邮箱地址")
            phone: str = Field(description="用户的手机号")

        # 3. agent 初始化
        agent = create_agent(
            model=model,
            response_format=ProviderStrategy(ContactInfo)
        )

        # 4. 调用
        response = agent.invoke({
            "messages": [
                HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：shkstart@atguigu.com，手机号：12345678912")
            ]
        })
        # rprint(response)

        for msg in response["messages"]:
            msg.pretty_print()
```

上面的会报错，换成gpt模型不会报错

② ToolStrategy

对于不支持原生结构化输出的模型，LangChain采用“ToolStrategy”工具调用的方式实现结构化输出。

此策略兼容绝大多数支持工具调用的现代模型，其核心原理是动态创建一个“虚拟工具”，该工具的输入参数对应着期望的数据结构。

当模型需要生成最终答案时，系统会引导模型“调用”这个虚拟工具，从而间接产生符合要求的数据。

举例：注意：deepseek-v4-flash 开启思考模式（reasoner）时，

```
** function_calling + tool_choice="required"
```

```
In [20]: from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)

# 2. Pydantic结构化方式定义
class ContactInfo(BaseModel):
    name: str = Field(description="姓名")
    email: str = Field(description="邮箱")
    phone: str = Field(description="电话")

# 3. 工具的定义（根据需要定义）
@tool
def search_tool(query: str) -> str:
    """这是一个搜索引擎。当大模型发现给定的上下文里缺少必要的联系人信息，
    需要去互联网上查询时，才会调用这个工具。
    """
    return f"搜索结果: 未找到关于 '{query}' 的更多额外信息。"

# 3. agent 初始化
agent = create_agent(
    model=model,
    tools=[search_tool],
    response_format=ToolStrategy(ContactInfo)
)

result = agent.invoke({
```

```

    "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
})

# print(result)
print(result["structured_response"])

```

name='John Doe' email='john@atguigu.com' phone='(010) 56253825'

③ type / AutoStrategy

官方没有参数列表或官方文档列出这种策略，但阅读源码可以看到。

```

In [ ]: ResponseFormat = ToolStrategy[SchemaT] | ProviderStrategy[SchemaT] |
AutoStrategy[SchemaT]
        """Union type for all supported response format strategies."""

```

当我们直接传入一个定义类型时，LangChain会自动包装为AutoStrategy，触发自动选择策略：如果模型支持原生结构化输出（如OpenAI、Anthropic Claude或xAI Grok），则优先使用ProviderStrategy；否则使用ToolStrategy。

举例1：

```

In [21]: from langchain.agents.structured_output import AutoStrategy
        from pydantic import BaseModel, Field
        from langchain.agents import create_agent

        class ContactInfo(BaseModel):
            """联系人信息"""
            name: str = Field(description="姓名")
            email: str = Field(description="邮箱")
            phone: str = Field(description="电话")

        agent = create_agent(
            model=model,
            response_format=AutoStrategy(ContactInfo)
        )

        result = agent.invoke({
            "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
        })

        # print(result)
        print(result["structured_response"])

```

name='John Doe' email='john@atguigu.com' phone='(010) 56253825'

特别注意：在LangChain 1.0及以上版本中，直接传递模式（如response_format=ContactInfo）不再支持，必须显式使用ToolStrategy或ProviderStrategy。（经过测试，目前LangChain 1.2版本还可使用）。

举例2：

```

In [22]: from pydantic import BaseModel, Field
        from langchain.agents import create_agent

```



```

class ContactInfo(BaseModel):
    """联系人信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="邮箱")
    phone: str = Field(description="电话")

agent = create_agent(
    model=model,
    response_format=ContactInfo # Auto-selects ProviderStrategy
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
})

# print(result)
print(result["structured_response"])

```

name='John Doe' email='john@atguigu.com' phone='(010) 56253825'

④ None

默认配置，表示不以结构化输出，以自然语言响应用户问题。

In [27]:

```

from pydantic import BaseModel, Field
from langchain.agents import create_agent

agent = create_agent(
    model=model,
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
})

# print(result)
print(result)

```

```
{'messages': [HumanMessage(content='联系人信息: John Doe,john@atguigu.com, (010) 56253825', additional_kwargs={}, response_metadata={}, id='b2d2e689-bf85-43fd-8afd-3beaa22afea8'), AIMessage(content='这是一个典型的联系人信息条目。根据您的需求，我可以为您整理成以下清晰、规范的格式，并附上使用建议。\\n\\n### 1. 标准格式（适用于名片、通讯录）\\n\\n`\\`\\`\\n姓名: John Doe\\n邮箱: john@atguigu.com\\n电话: (010) 56253825\\n`\\`\\`\\n\\n### 2. 国际化/紧凑格式（适用于简历、签名档）\\n\\n`\\`\\`\\nJohn Doe\\nE-mail: john@atguigu.com\\nTel: (010) 56253825\\n`\\`\\`\\n\\n### 3. 技术/数据格式（适用于数据库存储或API）\\n\\n如果您需要将其作为数据输入，通常建议拆分字段并标准化电话格式（去掉括号，使用国际区号）:\\n\\n`\\`\\`json\\n{\\n  "name": "John Doe",\\n  "email": "john@atguigu.com",\\n  "phone": "+86-10-56253825"\\n}\\n`\\`\\`\\n\\n---\\n\\n**补充说明（针对您提供的信息）:**\\n\\n* **姓名** : John Doe 是一个英文通用名，通常用于示例或占位。\\n* **邮箱后缀** : `atguigu.com` 表明这可能是与“尚硅谷”（atguigu）相关的邮箱。\\n* **电话格式** : `(010) 56253825` 是中国北京的区号格式，但缺少国际区号 `+86`。如果是用于国际联系，建议补充为 `+86-10-56253825`。\\n\\n**请问您需要我针对这些信息做进一步的操作吗？** 例如:\\n* 检测这个邮箱或电话是否有效？\\n* 将其转换为特定软件的导入格式（如 CSV）?', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 339, 'prompt_tokens': 24, 'total_tokens': 363, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 24}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_a18b46594c_prod0820_fp8_kvcache_20260402', 'id': '5dfdcf7c-9689-4a8e-b81d-a579175d5539', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ff485-de23-7dc0-a426-a36e30bceeaab-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 24, 'output_tokens': 339, 'total_tokens': 363, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}])}]}
```

总结：在实际大模型Agent开发场景中，如果使用到了结构化输出，推荐使用“ToolStrategy”策略，所以后续重点介绍这种策略方式结构化输出。

7.3 ToolStrategy使用详解

ToolStrategy通过工具调用（Tool Calling/Function Calling）实现结构化输出，所以LangChain会在消息列表末尾追加一条ToolMessage，让整个链路完整。但实际上没有实际的工具执行，这是一条伪消息。

ToolStrategy适用于任何支持工具调用的现代模型。

ToolStrategy的配置包含三个主要参数：

```
In [ ]: class ToolStrategy(Generic[SchemaT]):
        schema: type[SchemaT]
        tool_message_content: str | None
        handle_errors: Union[bool, str, type[Exception], tuple[type[Exception], ...], Callable[[Exception], str]]
```

- **schema**（必需参数）：与提供商策略的schema参数功能一致，支持Pydantic模型、TypedDict、JSON Schema、数据类(@dataclass)，同时还支持联合类型Union[类型1, 类型2]（允许模型根据输入内容选择最匹配的数据结构）。
- **tool_message_content**（可选参数）：用于自定义生成结构化输出时，会话历史中记录的提示信息。默认使用展示输出数据的标准响应语句。
- **handle_errors**（可选参数）：用于指定数据校验失败时的重试策略，默认值为True。

1) 结构化输出：schema参数

下面演示四种Schema进行结构化输出代码实现。

提前说明：因为涉及到不同的Schema在不同模型供应商下表现的支持力度不同

```
In [28]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model_ds = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)
```

```
In [29]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model_bailian = init_chat_model(
    model="deepseek-v4-flash-0731",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)
```

输出模式1：Pydantic类型

Pydantic类型的Schema支持数据验证，是优先推荐使用的方式。

举例1：

```
In [30]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")
```

```

agent = create_agent(
    model=model_ds,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，
手机号：12345678912")
    ]
})
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_uT2UMF50MfZjmzEFWgRp1485)

Call ID: call_00_uT2UMF50MfZjmzEFWgRp1485

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: name='小明' email='songhk@atguigu.com' phone='12345678912'

```

In [31]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，
手机号：12345678912")
    ]
})
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_0c47eda6bd7a4ec2bb892fcd)

Call ID: call_0c47eda6bd7a4ec2bb892fcd

Args:

email: songhk@atguigu.com

name: 小明

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: name='小明' email='songhk@atguigu.com' phone='12345678912'

观察日志可知，这种方式将结构化信息作为伪工具传递，显然使用了Function Calling方法。

举例2：

```
In [32]: from langchain_core.messages import SystemMessage
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.tools import tool
from rich import print as rprint

# 定义工具
@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串，例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串，包含客户姓名、等级、最近购买日期和累计消费
    """
    # 模拟数据库查询结果
    if "张三" in query.lower():
        return "客户记录：张三，VIP客户，最近购买日期：2026-01-15，累计消费：$15,000"
    elif "李四" in query.lower():
        return "客户记录：李四，普通客户，最近购买日期：2025-12-20，累计消费：$3,200"
    else:
        return f"关于客户{query}，无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称，例如 "张三" 或 "李四"

    Returns:
        str: 确认消息，包含已发送的客户名称
```

```

"""
return f"已向 {customer} 发送感谢邮件"

# 定义Pydantic Schema
class CustomerAnalysis(BaseModel):
    """客户分析报告"""
    customer_name: str = Field(None, description="客户姓名")
    customer_tier: Literal["潜在客户", "普通客户", "VIP客户", "流失风险"] = Field("潜在客户", description="客户等级,只能是潜在客户、普通客户、VIP客户或流失风险")
    recent_activity: str = Field(None, description="最近活动")
    spending_level: Literal["低", "中", "高"] = Field(None, description="消费水平")
    send_email: bool = Field(False, description="是否已发送感谢邮件")

# 创建智能体
agent = create_agent(
    model=model_ds,
    system_prompt=SystemMessage(content="""
        请分析指定客户的情况：
        1. 先搜索客户数据库了解最新情况
        2. 如果是VIP客户，则发送感谢邮件
        3. 基于搜索结果生成结构化分析报告
        4. 如果用户提问与客户记录无关或找不到客户信息，则返回空对象，不发送感谢邮件
    """),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(CustomerAnalysis)
)

# 执行分析
result = agent.invoke({
    "messages": [{"role": "user", "content": "请分析客户张三"}]
    # "messages": [{"role": "user", "content": "请分析客户李四"}]
    # "messages": [{"role": "user", "content": "请分析客户王五"}]
    # "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
# pprint(result)

if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)

```

customer_name='张三' customer_tier='VIP客户' recent_activity='最近购买日期：2026-01-15'
 spending_level='高' send_email=True

注意：

1. 如果是结构化输出，在系统提示词中最后提示结构化输出结果，如果提示词中先结构化输出结果（Agent已经执行完成），可能会导致一些工具不会再被调用。
2. 系统提示词中最后加入“未找到用户”时的处理提示，避免程序一直调用工具尝试查找对应用户信息。

输出模式2：TypedDict类型

TypedDict 允许为字典对象定义固定的键名和对应的值类型，是带有类型提示的字典结构。具体的：

1. TypedDict字段定义采用 Annotated[类型, 默认值, "描述"]格式
2. 可选字段使用 Optional包装，默认值在 Annotated 中指定
3. TypedDict 不支持运行时验证。

举例1：

```
In [34]: from typing import Annotated
from typing_extensions import TypedDict
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage

class ContactInfo(TypedDict):
    """用户的联系方式"""
    name: Annotated[str, ..., "用户姓名"]
    email: Annotated[str, ..., "用户邮箱地址"]
    phone: Annotated[str, ..., "用户的手机号"]

agent = create_agent(
    model=model_ds,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
    ]
})

for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])
```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_R1PORamTJzGmU7tAT6aA3564)

Call ID: call_00_R1PORamTJzGmU7tAT6aA3564

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: {'name': '小明', 'email': 'songhk@atguigu.com', 'phone': '12345678912'}

举例2：

```
In [37]: from langchain_core.messages import SystemMessage
from typing import Literal, Optional
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.tools import tool
```

```

# 定义工具
@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串, 例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串, 包含客户姓名、等级、最近购买日期和累计消费
    """
    # 模拟数据库查询结果
    if "张三" in query.lower():
        return "客户记录：张三, VIP客户, 最近购买日期：2026-01-15, 累计消费：$15,000"
    elif "李四" in query.lower():
        return "客户记录：李四, 普通客户, 最近购买日期：2025-12-20, 累计消费：$3,200"
    else:
        return f"关于客户{query}, 无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称, 例如 "张三" 或 "李四"

    Returns:
        str: 确认消息, 包含已发送的客户名称
    """
    return f"已向 {customer} 发送感谢邮件"

# 使用 TypedDict 定义客户分析报告 Schema
class CustomerAnalysis(TypedDict):
    """客户分析报告"""
    customer_name: Annotated[Optional[str], None, "客户姓名"]
    customer_tier: Annotated[Literal["潜在客户", "普通客户", "VIP客户", "流失风险"], "潜在客户", "客户等级"]
    recent_activity: Annotated[Optional[str], None, "最近活动"]
    spending_level: Annotated[Optional[Literal["低", "中", "高"]], None, "消费水平"]
    send_email: Annotated[bool, False, "是否已发送感谢邮件"]

# 创建智能体
agent = create_agent(
    model=model_ds,
    system_prompt=SystemMessage(content="""
    请分析指定客户的情况：
    "1. 先搜索客户数据库了解最新情况 "
    "2. 如果是VIP客户, 则发送感谢邮件 "
    "3. 基于搜索结果生成结构化分析报告 "
    "4. 如果用户提问与客户记录无关或找不到客户信息, 则返回空对象, 不发送感谢邮件"
    ""),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(CustomerAnalysis)
)

# 执行分析
result = agent.invoke({

```



```
"messages": [{"role": "user", "content": "请分析客户张三"}]
# "messages": [{"role": "user", "content": "请分析客户李四"}]
# "messages": [{"role": "user", "content": "请分析客户王五"}]
# "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
rprint("result:", result)
if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)
```

```

result:
{
  'messages': [
    HumanMessage(
      content='请分析客户张三',
      additional_kwargs={},
      response_metadata={},
      id='d1eae571-f150-4d1f-a1c5-a8898638e525'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 39,
          'prompt_tokens': 610,
          'total_tokens': 649,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512},
          'prompt_cache_hit_tokens': 512,
          'prompt_cache_miss_tokens': 98
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'a69ede21-d28f-49ce-91e8-10c41d512a34',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff4a8-cc05-7810-a853-2244598cedde-0',
      tool_calls=[
        {
          'name': 'search_customer_database',
          'args': {'query': '张三'},
          'id': 'call_00_ddF2VAA4u4rXg58AeiJv6899',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 610,
        'output_tokens': 39,
        'total_tokens': 649,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='客户记录：张三，VIP客户，最近购买日期：2026-01-15，累
计消费：$15,000',
      name='search_customer_database',
      id='ec426516-ed44-4b9a-a0eb-762f543f563b',
      tool_call_id='call_00_ddF2VAA4u4rXg58AeiJv6899'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 36,
          'prompt_tokens': 694,

```

```

        'total_tokens': 730,
        'completion_tokens_details': None,
        'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 640},
        'prompt_cache_hit_tokens': 640,
        'prompt_cache_miss_tokens': 54
    },
    'model_provider': 'deepseek',
    'model_name': 'deepseek-v4-flash',
    'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
    'id': '3f2bfc75-e38f-4d68-a8ec-06c184143932',
    'finish_reason': 'tool_calls',
    'logprobs': None
},
id='lc_run--019ff4a8-d0d1-7591-be77-a2b5abf66e5b-0',
tool_calls=[
    {
        'name': 'send_email',
        'args': {'customer': '张三'},
        'id': 'call_00_38yxRWwOZ4fNuTqNiwqg7733',
        'type': 'tool_call'
    }
],
invalid_tool_calls=[],
usage_metadata={
    'input_tokens': 694,
    'output_tokens': 36,
    'total_tokens': 730,
    'input_token_details': {'cache_read': 640},
    'output_token_details': {}
}
),
ToolMessage(
    content='已向 张三 发送感谢邮件',
    name='send_email',
    id='b7600850-57df-44db-abd3-3fe7c378d481',
    tool_call_id='call_00_38yxRWwOZ4fNuTqNiwqg7733'
),
AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 119,
            'prompt_tokens': 757,
            'total_tokens': 876,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 640},
            'prompt_cache_hit_tokens': 640,
            'prompt_cache_miss_tokens': 117
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'cf04ceda-f962-42e4-a10f-fa1c912c58ee',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    id='lc_run--019ff4a8-d51f-7c60-8ad3-c9e5654bf83f-0',
    tool_calls=[
        {

```

```

        'name': 'CustomerAnalysis',
        'args': {
            'customer_name': '张三',
            'customer_tier': 'VIP客户',
            'recent_activity': '最近购买日期：2026-01-15',
            'spending_level': '高',
            'send_email': True
        },
        'id': 'call_00_PXFX2LxbHTUIdhiOGXgb9193',
        'type': 'tool_call'
    },
    ],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 757,
        'output_tokens': 119,
        'total_tokens': 876,
        'input_token_details': {'cache_read': 640},
        'output_token_details': {}
    }
),
ToolMessage(
    content="Returning structured response:
{'customer_name': '张三', 'customer_tier': 'VIP客户',
'recent_activity': '最近购买日期：2026-01-15', 'spending_level': '高',
'send_email': True}",
    name='CustomerAnalysis',
    id='0f535ec4-ae9a-44e8-886d-d9d9e0d450d8',
    tool_call_id='call_00_PXFX2LxbHTUIdhiOGXgb9193'
)
],
'structured_response': {
    'customer_name': '张三',
    'customer_tier': 'VIP客户',
    'recent_activity': '最近购买日期：2026-01-15',
    'spending_level': '高',
    'send_email': True
}
}
{'customer_name': '张三', 'customer_tier': 'VIP客户', 'recent_activity': '最近购买日期：2026-01-15',
'spending_level': '高', 'send_email': True}

```

输出模式3：JsonSchema类型

JSON Schema是提供一个标准的 JSON Schema 字典来定义结构。适合需要与多种编程语言交互或进行复杂数据约束定义的场景。

举例1：

```

In [38]: from langchain.agents import create_agent
from langchain.messages import HumanMessage

json_schema = {
    "title": "ContactInfo",
    "description": "用户的联系方式",
    "type": "object",
    "properties": {
        "name": {
            "description": "用户姓名",

```

```

        "type": "string"
    },
    "email": {
        "description": "用户邮箱地址",
        "type": "string"
    },
    "phone": {
        "description": "用户的手机号",
        "type": "string"
    }
},
"required": [
    "name",
    "email",
    "phone"
]
}
agent = create_agent(
    model=model,
    response_format=ToolStrategy(json_schema)
)

response = agent.invoke(
    {
        "messages": [
            HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
        ]
    }
)
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_6zAEJTbWpDxqJvfXalhA5920)

Call ID: call_00_6zAEJTbWpDxqJvfXalhA5920

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: {'name': '小明', 'email': 'songhk@atguigu.com', 'phone': '12345678912'}

举例2：

```

In [39]: from langchain_core.messages import SystemMessage
        from typing import Literal, Optional
        from langchain.agents import create_agent
        from langchain.agents.structured_output import ToolStrategy
        from langchain.tools import tool

        # 定义工具

```

```

@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串, 例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串, 包含客户姓名、等级、最近购买日期和累计消费
    """
    # 模拟数据库查询结果
    if "张三" in query.lower():
        return "客户记录：张三, VIP客户, 最近购买日期：2026-01-15, 累计消费：$15,000"
    elif "李四" in query.lower():
        return "客户记录：李四, 普通客户, 最近购买日期：2025-12-20, 累计消费：$3,200"
    else:
        return f"关于客户{query}, 无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称, 例如 "张三" 或 "李四"

    Returns:
        str: 确认消息, 包含已发送的客户名称
    """
    return f"已向 {customer} 发送感谢邮件"

# 定义 JSON Schema 替代 Pydantic 模型
customer_analysis_schema = {
    "title": "CustomerAnalysis",
    "type": "object",
    "description": "客户分析报告",
    "properties": {
        "customer_name": {
            "type": "string",
            "default": "",
            "description": "客户姓名"
        },
        "customer_tier": {
            "type": "string",
            "enum": ["潜在客户", "普通客户", "VIP客户", "流失风险"],
            "default": "潜在客户",
            "description": "客户等级"
        },
        "recent_activity": {
            "type": "string",
            "default": "",
            "description": "最近活动"
        },
        "spending_level": {
            "type": "string",
            "enum": ["低", "中", "高"],
            "default": "低",
            "description": "消费水平"
        }
    }
},

```

```

    "send_email": {
        "type": "boolean",
        "default": False,
        "description": "是否已发送感谢邮件"
    }
},
# 所有字段都是必须输出的
"required": ["customer_name", "customer_tier", "recent_activity", "spending_level"]
}

# 创建智能体
agent = create_agent(
    model=model,
    system_prompt=SystemMessage(content="""
        请分析指定客户的情况：
        "1. 先搜索客户数据库了解最新情况 "
        "2. 如果是VIP客户，则发送感谢邮件 "
        "3. 基于搜索结果生成结构化分析报告 "
        "4. 如果用户提问与客户记录无关或找不到客户信息，则返回空对象，不发送感谢邮件"
    """),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(customer_analysis_schema)
)

# 执行分析
result = agent.invoke({
    "messages": [{"role": "user", "content": "请分析客户张三"}]
    # "messages": [{"role": "user", "content": "请分析客户李四"}]
    # "messages": [{"role": "user", "content": "请分析客户王五"}]
    # "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
# print("result:", result)
if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)

```

```
{'customer_name': '张三', 'customer_tier': 'VIP客户', 'recent_activity': '最近购买日期：2026-01-15', 'spending_level': '高', 'send_email': True}
```

注意以上代码中定义json_schema的时候指定的title, description, type, properties, required是遵循JSON Schema 规范的标准关键字，是固定写法。

输出模式4：@dataclass类型

@dataclass是Python 3.7引入的一个装饰器，用于简化数据存储类的定义。

举例1：

```
In [41]: from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.messages import HumanMessage
```

```
@dataclass
class ContactInfo:
    """用户的联系方式"""
```

```

name: str = Field(description="用户姓名")
email: str = Field(description="用户邮箱地址")
phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke(
    {
        "messages": [
            HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
        ]
    }
)
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_SO0tHhRZp6FnRrf9Ua6V1330)

Call ID: call_00_SO0tHhRZp6FnRrf9Ua6V1330

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: ContactInfo(name='小明', email='songhk@atguigu.com', phone='12345678912')

举例2：

```

In [42]: from langchain_core.messages import SystemMessage
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.tools import tool

# 定义工具
@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串，例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串，包含客户姓名、等级、最近购买日期和累计消费
    """

```



```

# 模拟数据库查询结果
if "张三" in query.lower():
    return "客户记录：张三，VIP客户，最近购买日期：2026-01-15，累计消费：$15,000"
elif "李四" in query.lower():
    return "客户记录：李四，普通客户，最近购买日期：2025-12-20，累计消费：$3,200"
else:
    return f"关于客户{query}，无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称，例如 "张三" 或 "李四"

    Returns:
        str: 确认消息，包含已发送的客户名称
    """
    return f"已向 {customer} 发送感谢邮件"

# 使用Dataclass定义Schema
@dataclass
class CustomerAnalysis:
    """客户分析报告"""
    customer_name: str = Field(None, description="客户姓名")
    customer_tier: Literal["潜在客户", "普通客户", "VIP客户", "流失风险"] = Field("潜在客户",
description="客户等级,只能是潜在客户、普通客户、VIP客户或流失风险")
    recent_activity: str = Field(None, description="最近活动")
    spending_level: Literal["低", "中", "高"] = Field(None, description="消费水平")
    send_email: bool = Field(False, description="是否已发送感谢邮件")

# 创建智能体
agent = create_agent(
    model=model,
    system_prompt=SystemMessage(content="""
    请分析指定客户的情况：
    "1. 先搜索客户数据库了解最新情况 "
    "2. 如果是VIP客户，则发送感谢邮件 "
    "3. 基于搜索结果生成结构化分析报告 "
    "4. 如果用户提问与客户记录无关或找不到客户信息，则返回空对象，不发送感谢邮件"
    """),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(CustomerAnalysis)
)

# 执行分析
result = agent.invoke({
    "messages": [{"role": "user", "content": "请分析客户张三"}]
    # "messages": [{"role": "user", "content": "请分析客户李四"}]
    # "messages": [{"role": "user", "content": "请分析客户王五"}]
    # "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
# print("result:", result)
if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)

```

CustomerAnalysis(customer_name='张三', customer_tier='VIP客户', recent_activity='最近购买日期：2026-01-15', spending_level='高', send_email=True)

多schema联合模式

ToolStrategy允许指定多个类型“ Union[类型1, 类型2] ”这种写法，LLM能够根据输入文本的内容，智能地选择最合适的一个数据模型（Schema）来生成结构化输出，但是最终会只有一种类型输出。

适用于根据不同输入内容，生成不同的结构化输出的场景，但是底层工具转换结构化输出只会转换成一种结构化类型输出。

```
In [43]: from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.messages import HumanMessage

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

class EventInfo(BaseModel):
    """事件详情"""
    event_name: str = Field(description="事件名称")
    date: str = Field(description="事件发生日期")

agent = create_agent(
    model=model,
    response_format=ToolStrategy(
        Union[ContactInfo, EventInfo]
    )
)
response = agent.invoke(
    {
        "messages": [
            HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：
shkstart@atguigu.com，手机号：12345678912")
        ]
    }
)
for msg in response["messages"]:
    msg.pretty_print()

print(response["structured_response"])
```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：shkstart@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_hwj7uGroYvmck7wc7Vk4935)

Call ID: call_00_hwj7uGroYvmck7wc7Vk4935

Args:

name: 小明

email: shkstart@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: name='小明' email='shkstart@atguigu.com'

phone='12345678912'

name='小明' email='shkstart@atguigu.com' phone='12345678912'

继续：

```
In [44]: response = agent.invoke(  
    {  
        "messages": [  
            HumanMessage("从这段话中抽取结构化信息：2026年高考报名人数突破1200万")  
        ]  
    }  
)  
for msg in response["messages"]:  
    msg.pretty_print()  
  
print(response["structured_response"])
```

===== Human Message =====

从这段话中抽取结构化信息：2026年高考报名人数突破1200万

===== Ai Message =====

Tool Calls:

EventInfo (call_00_qwSSmwulZIH9AdMvXhU21700)

Call ID: call_00_qwSSmwulZIH9AdMvXhU21700

Args:

event_name: 高考报名

date: 2026

===== Tool Message =====

Name: EventInfo

Returning structured response: event_name='高考报名' date='2026'

event_name='高考报名' date='2026'

2) 自定义工具消息：tool_message_content参数

如果采用ToolStrategy策略处理结构化输出时，LangChain会在消息列表末尾追加一条Tool_message，让整个链路完整。但实际上没有实际的工具执行，这是一条伪消息。

我们可以通过ToolStrategy的 tool_message_content 参数定制其消息内容，将指定的内容写入对话历史的提示信息，这样做的好处如下：

1. 在最终用户可见的对话流中，使用更自然的消息替代原始数据。
2. 用简短的确认信息替代可能很长的数据块，减少token消耗。

举例：

当不设置 `tool_message_content` 时，模型收到的 `ToolMessage` 里就包含了像 `{'name': '张三', 'email': 'zhangsan@email.com'... ..}` 这样的具体数据。当设置了 `tool_message_content` 时，模型收到的 `ToolMessage` 只是一个预定义的确认信息，如“ 格式化输出成功！”。这种方式节省了上下文窗口的令牌消耗，并且让对话流对最终用户更友好。

说明：无论 `tool_message_content` 如何设置，成功提取的结构化数据最终都会正确存入 `result["structured_response"]` 返回，自定义消息仅影响对话历史中的一条记录。

举例：默认情况

```
In [2]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model_bailian = init_chat_model(
    model="deepseek-v4-flash-0731",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)
```

```
In [4]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from rich import print as rprint

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(ContactInfo)
)
response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
    ]
})
rprint(response)
```

```

{
  'messages': [
    HumanMessage(
      content='从这段话中抽取结构化信息：小明的邮箱地址为：
songhk@atguigu.com, 手机号：12345678912',
      additional_kwargs={},
      response_metadata={},
      id='61f5e301-da38-4ab1-a442-d9e2d2daaf92'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 83,
          'prompt_tokens': 344,
          'total_tokens': 427,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcmpl-e97851c4-181c-9cf6-9240-
9742fc8bd0e4',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff625-161d-7412-9b84-acabfdc6bab1-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'songhk@atguigu.com', 'name':
'小明', 'phone': '12345678912'},
          'id': 'call_128ddd40c68541bc974cda1a',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 344,
        'output_tokens': 83,
        'total_tokens': 427,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content="Returning structured response: name='小明'
email='songhk@atguigu.com' phone='12345678912'",
      name='ContactInfo',
      id='0ebec4c8-3a3a-4eb0-b618-e0d409cbd332',
      tool_call_id='call_128ddd40c68541bc974cda1a'
    )
  ],
  'structured_response': ContactInfo(name='小明',
email='songhk@atguigu.com', phone='12345678912')
}

```

自定义tool_message_content

```
In [6]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from rich import print as rprint

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(ContactInfo,tool_message_content="已成功抽取信息")
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：
songhk@atguigu.com，手机号：12345678912")
    ]
})
rprint(response)
```

```

{
  'messages': [
    HumanMessage(
      content='从这段话中抽取结构化信息：小明的邮箱地址为：
songhk@atguigu.com, 手机号：12345678912',
      additional_kwargs={},
      response_metadata={},
      id='44d4bb65-383f-431a-8789-6502d342ae44'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 83,
          'prompt_tokens': 344,
          'total_tokens': 427,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-09db813d-3813-9118-ac4d-
65bdf58dceea',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff627-829f-7650-b200-ad2b83d0ba1b-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'songhk@atguigu.com', 'name':
'小明', 'phone': '12345678912'},
          'id': 'call_8d5dde27ed6d4d81b7782a8f',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 344,
        'output_tokens': 83,
        'total_tokens': 427,
        'input_token_details': {'cache_read': 256},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='已成功抽取信息',
      name='ContactInfo',
      id='e5419066-0b4e-4372-9f34-d12e91e10710',
      tool_call_id='call_8d5dde27ed6d4d81b7782a8f'
    )
  ],
  'structured_response': ContactInfo(name='小明',
email='songhk@atguigu.com', phone='12345678912')
}

```

3) 错误处理：handle_errors参数

受限于模型能力，大模型输出的内容可能并不符合格式要求，ToolStrategy通过其handle_errors参数提供了结构化过程错误处理策略，以下是主要的几种方式及其用途：

- **handle_errors=True**：LangChain默认方式，捕获所有异常，并使用LangChain内置的、信息明确的错误消息模板提示模型重试，确保最终能得到符合预定格式的有效数据。适用于大多数希望自动处理错误的通用场景。
- **handle_errors=False**：关闭自动重试机制，任何异常都会直接抛出，会中断程序运行。
- **handle_errors="自定义字符串"**：捕获所有异常，但使用开发者预设的固定字符串作为错误消息。适用于需要统一、友好的用户提示，或进行特定业务引导的场景。
- **handle_errors=ExceptionType**：仅捕获指定类型(如ValueError) 或元组中的异常类型并进行重试，其他异常直接抛出。适用于需要精准控制，只对特定错误进行重试的场景。
- **handle_errors=callable**：灵活性最高的方式，使用开发者自定义的函数来处理异常，可根据不同的异常类型返回差异化的提示信息。适用于需要复杂、精细化错误处理的场景。

情况1：设置为True/False/固定字符串

设计思路：模型对于单条信息的格式化输出请求，输出了多个工具调用请求。也称为多结构化输出错误。

```
In [7]: from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from rich import print as rprint

class ContactInfo(BaseModel):
    """个人联系信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="电子邮箱")

class EventDetails(BaseModel):
    """活动详情"""
    event_name: str = Field(description="活动名称")
    date: str = Field(description="活动日期")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(
        Union[ContactInfo, EventDetails],
        tool_message_content="提取完成！",
        handle_errors=True
        #handle_errors="请检查输入数据"
    )
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": f"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
    }]
})
```



```
})
```

```
rprint(result)
```

```

{
  'messages': [
    HumanMessage(
      content='请提取以下文本中内容：姓名：张三，电子邮箱：
zhang3@atguigu.com，活动名称：公司年会，活动日期：
2026-07-15',
      additional_kwargs={},
      response_metadata={},
      id='be4dc1c3-f151-4ca1-ba23-7004e19add12'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 118,
          'prompt_tokens': 404,
          'total_tokens': 522,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-b69a15f5-53f5-940d-a946-
92e0bef7b71c',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff642-1fc4-7040-a704-d0148803cc5d-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
          'id': 'call_22511cd5bac5413fbbb4dd8e',
          'type': 'tool_call'
        },
        {
          'name': 'EventDetails',
          'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
          'id': 'call_3e49282fa80b4242a3129f59',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 404,
        'output_tokens': 118,
        'total_tokens': 522,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='Error: Model incorrectly returned multiple
structured responses (ContactInfo, EventDetails)
when only one is expected.\n Please fix your mistakes.',
      name='ContactInfo',
      id='cbf7a7b0-9726-4f1d-9f37-273256d39144',
      tool_call_id='call_22511cd5bac5413fbbb4dd8e'
    )
  ]
}

```

```

    ),
    ToolMessage(
        content='Error: Model incorrectly returned multiple
structured responses (ContactInfo, EventDetails)
when only one is expected.\n Please fix your mistakes.',
        name='EventDetails',
        id='39dee023-bfc7-421a-b4ec-09a30369f657',
        tool_call_id='call_3e49282fa80b4242a3129f59'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 65,
                'prompt_tokens': 593,
                'total_tokens': 658,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-93756ee9-1294-98c2-9123-
fbb9fb4439ff',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff642-23c6-7c91-b038-b76408dcd42-0',
        tool_calls=[
            {
                'name': 'ContactInfo',
                'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
                'id': 'call_1bc7dc1e234e42b1a61fc1bd',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 593,
            'output_tokens': 65,
            'total_tokens': 658,
            'input_token_details': {'cache_read': 256},
            'output_token_details': {}
        }
    ),
    ToolMessage(
        content='提取完成!',
        name='ContactInfo',
        id='67b7cc47-3815-441a-b62f-df1bd302c58d',
        tool_call_id='call_1bc7dc1e234e42b1a61fc1bd'
    )
],
    'structured_response': ContactInfo(name='张三',
email='zhang3@atguigu.com')
}

```

1) handle_errors设置为True

运行后结果如下：

ToolMessage(content='Error: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected.\n Please fix your mistakes.',

模型接收到错误反馈后，再次生成新的调用请求，直至成功或达到内部的最大重试次数。ToolStrategy中自定义的tool_message_content控制的是成功后的消息内容，不影响错误消息内容。

```
In [ ]: from pydantic import BaseModel, Field
        from typing import Union
        from langchain.agents import create_agent
        from langchain.agents.structured_output import ToolStrategy
        from rich import print as rprint

        class ContactInfo(BaseModel):
            """个人联系信息"""
            name: str = Field(description="姓名")
            email: str = Field(description="电子邮箱")

        class EventDetails(BaseModel):
            """活动详情"""
            event_name: str = Field(description="活动名称")
            date: str = Field(description="活动日期")

        agent = create_agent(
            model=model_bailian,
            response_format=ToolStrategy(
                Union[ContactInfo, EventDetails],
                tool_message_content="提取完成！",
                handle_errors=False
                #handle_errors="请检查输入数据"
            )
        )

        result = agent.invoke({
            "messages":[{
                "role":"user",
                "content":"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
            }]
        })

        rprint(result)
```

2) handle_errors设置为False

运行后结果如下：MultipleStructuredOutputsError: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected. During task with name 'model' and id '91685c0f-9b1f-9d38-545f-839462aca4e9'

3) handle_errors设置为“请检查输入数据”

运行后结果如下：

```
In [10]: from pydantic import BaseModel, Field
         from typing import Union
```

```

from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from rich import print as rprint

class ContactInfo(BaseModel):
    """个人联系信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="电子邮箱")

class EventDetails(BaseModel):
    """活动详情"""
    event_name: str = Field(description="活动名称")
    date: str = Field(description="活动日期")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(
        Union[ContactInfo, EventDetails],
        tool_message_content="提取完成！",
        handle_errors="请检查输入数据"
    )
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": f"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
    }]
})

rprint(result)

```

```

{
  'messages': [
    HumanMessage(
      content='请提取以下文本中内容：姓名：张三，电子邮箱：
zhang3@atguigu.com，活动名称：公司年会，活动日期：
2026-07-15',
      additional_kwargs={},
      response_metadata={},
      id='671250a4-1662-4f78-945a-1ea423af7bf7'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 118,
          'prompt_tokens': 404,
          'total_tokens': 522,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-bddefd48-968c-975a-a4e3-
c1a5bbd8be34',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff646-021b-72c0-9ca2-23968458471c-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
          'id': 'call_c4c0c568ff514df58078a41b',
          'type': 'tool_call'
        },
        {
          'name': 'EventDetails',
          'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
          'id': 'call_0ba1d4cbd2b0462592e06fd6',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 404,
        'output_tokens': 118,
        'total_tokens': 522,
        'input_token_details': {'cache_read': 256},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='请检查输入数据',
      name='ContactInfo',
      id='aa9bbb37-b9e7-4a4b-9c35-571c8e7ef0c7',
      tool_call_id='call_c4c0c568ff514df58078a41b'
    ),
    ToolMessage(

```

```

        content='请检查输入数据',
        name='EventDetails',
        id='bbf93ddb-7a2f-46f3-8872-2f6915269d58',
        tool_call_id='call_0ba1d4cbd2b0462592e06fd6'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 118,
                'prompt_tokens': 551,
                'total_tokens': 669,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcpl-56095e45-d854-936c-bf5e-fbcdca36d789',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff646-05bd-7ef1-a4b8-98c3cf21af5f-0',
        tool_calls=[
            {
                'name': 'ContactInfo',
                'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
                'id': 'call_bb5f0c1e366349ffa69ed982',
                'type': 'tool_call'
            },
            {
                'name': 'EventDetails',
                'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
                'id': 'call_b54e1faf2c3043248a0fdc76',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 551,
            'output_tokens': 118,
            'total_tokens': 669,
            'input_token_details': {'cache_read': 256},
            'output_token_details': {}
        }
    ),
    ToolMessage(
        content='请检查输入数据',
        name='ContactInfo',
        id='d0d8a1b9-a0d0-4cb5-bcce-14145743d709',
        tool_call_id='call_bb5f0c1e366349ffa69ed982'
    ),
    ToolMessage(
        content='请检查输入数据',
        name='EventDetails',
        id='0c8aa974-9a3b-49d8-a8d2-48a02d954275',
        tool_call_id='call_b54e1faf2c3043248a0fdc76'
    ),

```

```

AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 118,
            'prompt_tokens': 698,
            'total_tokens': 816,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcmpl-6b8fbbbd-ed94-95eb-a419-2f799d7ae64b',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    id='lc_run--019ff646-090f-77f1-8846-4d0a97188f2d-0',
    tool_calls=[
        {
            'name': 'ContactInfo',
            'args': {'email': 'zhang3@atguigu.com', 'name': '张三'},
            'id': 'call_b8e3e42bcc704af18b09903e',
            'type': 'tool_call'
        },
        {
            'name': 'EventDetails',
            'args': {'date': '2026-07-15', 'event_name': '公司年会'},
            'id': 'call_04c2267127fc45248e7a59d1',
            'type': 'tool_call'
        }
    ],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 698,
        'output_tokens': 118,
        'total_tokens': 816,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {}
    }
),
ToolMessage(
    content='请检查输入数据',
    name='ContactInfo',
    id='5e584269-8f12-480d-a3d8-35acc9b8c91b',
    tool_call_id='call_b8e3e42bcc704af18b09903e'
),
ToolMessage(
    content='请检查输入数据',
    name='EventDetails',
    id='44d62d17-c78b-4fdf-8975-a1666306fa03',
    tool_call_id='call_04c2267127fc45248e7a59d1'
),
AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {

```



```

        'completion_tokens': 118,
        'prompt_tokens': 845,
        'total_tokens': 963,
        'completion_tokens_details': None,
        'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
    },
    'model_provider': 'openai',
    'model_name': 'deepseek-v4-flash-0731',
    'system_fingerprint': None,
    'id': 'chatcmpl-99cb3d82-5d83-966a-98c1-
1908752b5186',
    'finish_reason': 'tool_calls',
    'logprobs': None
},
id='lc_run--019ff646-0c94-76d2-8a9e-7f4ba37b48cc-0',
tool_calls=[
    {
        'name': 'EventDetails',
        'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
        'id': 'call_66a7299eb42043848e4acfa0',
        'type': 'tool_call'
    },
    {
        'name': 'ContactInfo',
        'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
        'id': 'call_27e12910614f4a678389cf95',
        'type': 'tool_call'
    }
],
invalid_tool_calls=[],
usage_metadata={
    'input_tokens': 845,
    'output_tokens': 118,
    'total_tokens': 963,
    'input_token_details': {'cache_read': 512},
    'output_token_details': {}
}
),
ToolMessage(
    content='请检查输入数据',
    name='EventDetails',
    id='6d9e6ec0-d62b-4ecf-adae-2d173efe2817',
    tool_call_id='call_66a7299eb42043848e4acfa0'
),
ToolMessage(
    content='请检查输入数据',
    name='ContactInfo',
    id='e5a8e6f6-04bd-4590-b78d-eb66fe50be29',
    tool_call_id='call_27e12910614f4a678389cf95'
),
AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 118,
            'prompt_tokens': 992,
            'total_tokens': 1110,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens': None,

```

```

'cached_tokens': 768}
    },
    'model_provider': 'openai',
    'model_name': 'deepseek-v4-flash-0731',
    'system_fingerprint': None,
    'id': 'chatcmpl-a59cd824-b38f-9280-bf7b-
f174934dc281',
    'finish_reason': 'tool_calls',
    'logprobs': None
  },
  id='lc_run--019ff646-0fc6-7381-bf94-5d24ce3dd612-0',
  tool_calls=[
    {
      'name': 'EventDetails',
      'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
      'id': 'call_34fab19cc5994aebbddf0d8f',
      'type': 'tool_call'
    },
    {
      'name': 'ContactInfo',
      'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
      'id': 'call_1e4c607d79f24ca99fb7f727',
      'type': 'tool_call'
    }
  ],
  invalid_tool_calls=[],
  usage_metadata={
    'input_tokens': 992,
    'output_tokens': 118,
    'total_tokens': 1110,
    'input_token_details': {'cache_read': 768},
    'output_token_details': {}
  }
),
ToolMessage(
  content='请检查输入数据',
  name='EventDetails',
  id='dcbfea5e-4dc4-4310-91ae-f1c35b4b47cb',
  tool_call_id='call_34fab19cc5994aebbddf0d8f'
),
ToolMessage(
  content='请检查输入数据',
  name='ContactInfo',
  id='e276c498-7c11-4eb5-9a1a-08c27984bf75',
  tool_call_id='call_1e4c607d79f24ca99fb7f727'
),
AIMessage(
  content='',
  additional_kwargs={'refusal': None},
  response_metadata={
    'token_usage': {
      'completion_tokens': 65,
      'prompt_tokens': 1139,
      'total_tokens': 1204,
      'completion_tokens_details': None,
      'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
    },
    'model_provider': 'openai',
    'model_name': 'deepseek-v4-flash-0731',
    'system_fingerprint': None,

```

```

        'id': 'chatcmpl-81c83be6-8feb-90b1-a24a-
94e812e41b5f',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    id='lc_run--019ff646-135f-7c13-b6f6-0d5aa2a8df54-0',
    tool_calls=[
        {
            'name': 'ContactInfo',
            'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
            'id': 'call_52e97821019c496b9e74ffc8',
            'type': 'tool_call'
        }
    ],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 1139,
        'output_tokens': 65,
        'total_tokens': 1204,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {}
    }
),
ToolMessage(
    content='提取完成!',
    name='ContactInfo',
    id='be18fc6b-907e-4ff8-be23-f3fbc7f430d8',
    tool_call_id='call_52e97821019c496b9e74ffc8'
),
    ],
    'structured_response': ContactInfo(name='张三',
email='zhang3@atguigu.com')
}

```

以上代码中注意如下几点：

1. ToolStrategy允许指定多个类型“Union[ContactInfo, EventDetails]”这种写法，但是最终只会转换成一种结构化类型输出。
2. 当ToolStrategy通过“Union[ContactInfo, EventDetails]”指定多个类型时，在内部调用生成结构化类型工具会报错。此时：
 - handle_errors=True（默认值）开始发挥作用，系统会生成一个ToolMessage，明确告诉LLM“Error: Model incorrectly returned multiple structured responses (ContactInfo,EventDetails) when only one is expected.”，大模型收到这个精准的反馈后，会重新进行推理，最终选择并输出一个最符合要求的Schema。
 - 如果handle_errors 设置为False，执行代码过程直接报错。
3. 当格式化输出有错误时，Agent内部会进行工具调用重试，直到符合要求格式化输出前，可能会进行多次重试。

情况2：设置为指定异常类型

默认情况下，LangChain会处理结构化输出处理时抛出的两类异常：

1. MultipleStructuredOutputsError

多结构化输出错误，当返回的工具调用请求数量大于1时，抛出该异常，默认情况下